

Memoria

Gestione della Memoria

Introduzione all'allocazione dinamica della memoria con new e delete in C++.

Come funziona la memoria in C++?

Immagina di organizzare una festa. Non sai quante persone verranno. Se prenoti 10 sedie prima, e vengono in 50, hai un problema. Se ne prenoti 200, e vengono in 10, hai sprecato spazio.

La **memoria dinamica** risolve questo problema: puoi "prenotare" esattamente la quantità di memoria che ti serve, nel momento in cui sai quanta ne hai bisogno.

Due tipi di memoria

In C++, la memoria funziona in due modi diversi:

Memoria automatica (stack): le variabili normali. Il computer le crea quando entri in una funzione e le distrugge da solo quando esci. La quantità deve essere nota in anticipo.

```
int x = 5;           // creato automaticamente, distrutto automaticamente
int arr[10];         // array di dimensione fissa – deve essere nota prima
```

Memoria dinamica (heap): tu decidi quanta memoria serve, nel momento in cui ti serve. Devi gestirla tu — compresa la "restituzione" quando hai finito.

```
int* ptr = new int;    // crei uno spazio nella memoria heap
int* arr = new int[100]; // un array di 100 interi – deciso a runtime
```

Pensa all'heap come a un magazzino enorme. Puoi affittare uno spazio, usarlo, e poi restituirlo. Se non lo restituisci, rimane occupato inutilmente.

Allocare memoria: **new**

L'operatore **new** affitta uno spazio nell'heap e ti restituisce l'indirizzo (sotto forma di puntatore):

```
// Alloca spazio per un singolo intero
int* ptr = new int;
*ptr = 42;
cout << *ptr << endl;    // 42

// Alloca e inizializza subito
int* ptr2 = new int(100);
cout << *ptr2 << endl;    // 100

// Alloca un array di 5 interi
int* arr = new int[5];
arr[0] = 10;
arr[1] = 20;
// ...
```

Liberare memoria: **delete**

Ogni volta che usi **new**, devi usare **delete** quando hai finito. Altrimenti la memoria rimane "affittata" per sempre — si chiama **memory leak** (perdita di memoria):

```
int* ptr = new int(42);
cout << *ptr << endl;    // 42

delete ptr;               // "restituisce" la memoria affittata
ptr = nullptr;            // buona abitudine: azzera il puntatore dopo delete
```

Per un array allocato con **new[]**, usa **delete[]** (con le parentesi quadre):

```
int* arr = new int[10];
// ... usa arr ...
delete[] arr;             // IMPORTANTE: delete[] e non solo delete
arr = nullptr;
```

Il memory leak: la perdita di memoria

Se dimentichi il `delete`, la memoria resta occupata finché il programma non chiude:

```
// Ogni volta che chiami questa funzione, perdi 4 byte di memoria
void funzione() {
    int* ptr = new int(42);
    // ... usa ptr ...
    // manca delete ptr! - il ptr viene distrutto ma la memoria no
}

// Se la chiami mille volte, hai perso 4000 byte senza saperlo
for (int i = 0; i < 1000; i++) {
    funzione();
}
```

In un programma piccolo non si nota. In un programma che gira per ore o giorni, i leak si accumulano fino a esaurire la RAM.

Il dangling pointer: il puntatore "fantasma"

Dopo aver chiamato `delete`, il puntatore esiste ancora ma punta a memoria che non è più tua. Usarlo è un errore grave:

```
int* ptr = new int(42);
delete ptr;
// ptr è ora un "puntatore fantasma": punta a memoria liberata

cout << *ptr;    // ERRORE: comportamento non definito (possibile crash)

// Soluzione: dopo delete, metti sempre nullptr
delete ptr;
ptr = nullptr;

if (ptr != nullptr) {
    cout << *ptr;    // non verrà mai eseguito, ma è sicuro
}
```

Esempio pratico: array di dimensione variabile

Il caso più comune in cui serve la memoria dinamica è quando non sai in anticipo quanti elementi hai bisogno:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Quanti numeri vuoi inserire? ";
    cin >> n;

    // Alloca esattamente n interi – deciso a runtime!
    int* numeri = new int[n];

    // Inserisci i numeri
    for (int i = 0; i < n; i++) {
        cout << "Numero " << (i + 1) << ": ";
        cin >> numeri[i];
    }

    // Calcola la media
    int somma = 0;
    for (int i = 0; i < n; i++) {
        somma += numeri[i];
    }
    cout << "Media: " << (double)somma / n << endl;

    // Libera la memoria!
    delete[] numeri;
    numeri = nullptr;

    return 0;
}
```

La soluzione moderna: puntatori smart

Gestire `new` e `delete` manualmente è complicato e soggetto a errori. In C++ moderno si usano i **puntatori smart**, che si occupano di liberare la memoria automaticamente quando non serve più:

```
#include <memory>
using namespace std;

// unique_ptr: possiede la memoria e la libera automaticamente
unique_ptr<int> ptr = make_unique<int>(42);
cout << *ptr << endl;    // 42
// Non serve delete: viene chiamato automaticamente quando ptr esce dal
// blocco

// shared_ptr: più puntatori condividono la stessa memoria
shared_ptr<int> sptr = make_shared<int>(100);
```

I puntatori smart eliminano quasi tutti i problemi di memory leak e dangling pointer.

La soluzione ancora più semplice: **vector**

Per gli array dinamici, la scelta migliore è usare **vector** dalla libreria standard. Gestisce la memoria per te:

```
#include <vector>
using namespace std;

// Invece di:
int* arr = new int[n];
// ... usa arr ...
delete[] arr;

// Usa semplicemente:
vector<int> arr(n);
// ... usa arr ...
// nessun delete! viene liberato automaticamente
```

Regole pratiche

- Se usi **new**, usa sempre **delete** nella stessa funzione (o usa uno smart pointer)
- Se usi **new[]**, usa sempre **delete[]** (non **delete**)
- Dopo ogni **delete**, imposta il puntatore a **nullptr**
- Quando puoi, usa **vector** invece di array dinamici manuali

- Per oggetti singoli, usa `make_unique` invece di `new`

Puntatori

Il concetto di puntatori in C++, come funzionano e come usarli.

Cos'è un puntatore?

Immagina di avere un quaderno con le istruzioni per trovare un tesoro. Il quaderno non contiene il tesoro — contiene l'**indirizzo** dove trovarlo.

Un **puntatore** funziona allo stesso modo: non contiene un valore direttamente, ma contiene la **posizione in memoria** dove quel valore si trova.

Questo ti permette di fare cose che altrimenti non potresti fare, come modificare una variabile dall'interno di una funzione.

La memoria del computer come una via di appartamenti

Pensa alla RAM come a una lunghissima via con migliaia di appartamenti. Ogni appartamento ha un **numero civico** (l'indirizzo) e può contenere un valore.

Quando dichiari `int x = 42;`, il computer affitta un appartamento per `x` e ci mette dentro il numero 42.

Un puntatore è come un bigliettino su cui scrivi il numero civico di quell'appartamento. Con il bigliettino, puoi andare a vedere cosa c'è dentro — o addirittura cambiarlo.

Come vedere l'indirizzo di una variabile

Usa l'operatore `&` davanti a una variabile per leggere il suo indirizzo:

```
int x = 42;
cout << x << endl;    // stampa 42 (il valore)
cout << &x << endl;   // stampa qualcosa come 0x7fff5c1a2b40 (l'indirizzo)
```

Quell'indirizzo strano con lettere e numeri è scritto in **esadecimale** — è il numero del "cassetto" nella RAM.

Dichiarare un puntatore

Si usa il simbolo `*` dopo il tipo:

```
int* ptr;          // puntatore a int (un bigliettino con l'indirizzo di un
intero)
double* dptr;      // puntatore a double
char* cptr;        // puntatore a char
```

Assegnare un indirizzo al puntatore

Per far "puntare" il puntatore a una variabile, assegnagli il suo indirizzo con `&` :

```
int x = 42;
int* ptr = &x;    // ptr contiene l'indirizzo di x (il "numero civico" di
x)

cout << ptr << endl;    // stampa l'indirizzo di x
cout << *ptr << endl;    // stampa 42 (il valore dentro quell'indirizzo)
```

Leggere e modificare il valore: l'operatore `*`

Mettere `*` davanti a un puntatore significa "vai all'indirizzo e leggi (o scrivi) il valore lì dentro". Si chiama **dereferenziazione**.

```
int x = 42;
int* ptr = &x;

cout << *ptr << endl;    // legge il valore di x → stampa 42

*ptr = 100;              // scrive un nuovo valore nella posizione di x
cout << x << endl;        // x è cambiata! stampa 100
```

È come andare all'appartamento indicato sul bigliettino e sostituire il contenuto.

Riepilogo dei simboli

```
int x = 42;
int* ptr = &x;

// & davanti a una variabile = "dammi l'indirizzo di questa variabile"
cout << &x << endl;    // indirizzo di x

// * nella dichiarazione = "questa variabile è un puntatore"
int* ptr = &x;          // ptr è un puntatore a int

// * davanti a un puntatore = "vai all'indirizzo e leggi/scrivi il valore"
cout << *ptr << endl;   // il valore a quell'indirizzo (42)
```

Il puntatore nullo

Se un puntatore non deve puntare a niente, assegnagli `nullptr` :

```
int* ptr = nullptr;    // non punta a nessun indirizzo

// Prima di usare un puntatore, controlla che non sia nullo!
if (ptr != nullptr) {
    cout << *ptr;    // sicuro solo se ptr non è nullptr
}
```

Regola d'oro: non usare mai un puntatore nullo senza averlo controllato prima — causerebbe il crash del programma.

Passare variabili alle funzioni tramite puntatore

Il caso d'uso più utile dei puntatori è permettere a una funzione di modificare una variabile che si trova fuori da essa:


```

// La funzione riceve il "bigliettino" con l'indirizzo di x
void raddoppia(int* ptr) {
    *ptr = *ptr * 2;    // va all'indirizzo e modifica il valore lì
}

int main() {
    int x = 5;
    cout << x << endl;    // 5

    raddoppia(&x);        // passa l'indirizzo di x (il "bigliettino")

    cout << x << endl;    // 10 - x è stata modificata dalla funzione!
    return 0;
}

```

Senza i puntatori, la funzione riceverebbe solo una **copia** di `x` e le modifiche andrebbero perse.

Puntatori e array

Il nome di un array è già un puntatore al suo primo elemento. Puoi muoverti tra gli elementi sommando numeri all'indirizzo:

```

int numeri[] = {10, 20, 30, 40, 50};
int* ptr = numeri;    // punta al primo elemento (numeri[0])

cout << *ptr << endl;    // 10 (primo elemento)
cout << *(ptr + 1) << endl;    // 20 (secondo elemento)
cout << *(ptr + 2) << endl;    // 30 (terzo elemento)

// Equivalente all'accesso normale con []
cout << ptr[0] << endl;    // 10
cout << ptr[1] << endl;    // 20

```

Esempio pratico: scambiare due variabili

```
#include <iostream>
using namespace std;

// Riceve i bigliettini con gli indirizzi di a e b
void scambia(int* a, int* b) {
    int temp = *a;    // salva il valore di a in temp
    *a = *b;          // copia il valore di b in a
    *b = temp;        // copia temp (vecchio a) in b
}

int main() {
    int x = 10, y = 20;
    cout << "Prima: x=" << x << ", y=" << y << endl;

    scambia(&x, &y);    // passa gli indirizzi di x e y

    cout << "Dopo:  x=" << x << ", y=" << y << endl;
    return 0;
}
```

Output:

```
Prima: x=10, y=20
Dopo:  x=20, y=10
```

Errori comuni da evitare

```
int* ptr;        // NON inizializzato: contiene un indirizzo casuale,
                 // pericoloso!
*ptr = 42;       // ERRORE: modifica una zona di memoria a caso → crash

int* ptr2 = nullptr;
*ptr2 = 42;      // ERRORE: dereferenziare nullptr → crash garantito
```

In C++ moderno, si preferisce usare i **riferimenti** quando possibile (vedi la sezione successiva) e i **puntatori smart** per la memoria dinamica.

Riferimenti

Come usare i riferimenti in C++ per creare alias alle variabili.

Cos'è un riferimento?

Supponi di chiamarti "Marco" ma tutti gli amici ti chiamano "Max". Sei sempre la stessa persona — solo con due nomi diversi.

Un **riferimento** in C++ funziona così: è un secondo nome per la stessa variabile. Se modifichi il valore tramite un nome, l'altro nome vede subito la modifica — perché si riferiscono alla stessa cosa.

I riferimenti sono più semplici e sicuri dei puntatori, e si usano moltissimo in C++.

Come creare un riferimento

Si usa `&` dopo il tipo nella dichiarazione:

```
int x = 42;
int& ref = x;    // ref è un altro nome per x

cout << x << endl;    // 42
cout << ref << endl;  // 42 (stessa variabile, stesso valore)

ref = 100;        // modifica il valore tramite ref
cout << x << endl;    // 100 – anche x è cambiata!
```

`ref` e `x` puntano alla stessa casella in memoria. Cambiare uno cambia l'altro.

Differenze tra riferimenti e puntatori

Caratteristica	Riferimento (&)	Puntatore (*)
Come si usa	<code>ref</code> (direttamente)	<code>*ptr</code> (con la stella)
Può essere nullo?	No	Sì (<code>nullptr</code>)
Può cambiare a cosa si riferisce?	No	Sì
Deve essere inizializzato subito?	Sì	No (ma è pericoloso)

I riferimenti sono più comodi e sicuri: una volta creati, puntano sempre alla stessa variabile.

Il caso d'uso principale: funzioni che modificano variabili

Normalmente, quando passi una variabile a una funzione, la funzione ne riceve una **copia**. Le modifiche alla copia non cambiano l'originale:

```
void raddoppiaValore(int x) {  
    x = x * 2;    // modifica solo la copia  
}  
  
int main() {  
    int numero = 5;  
    raddoppiaValore(numero);  
    cout << numero << endl;    // 5 - non è cambiato!  
}
```

Con un riferimento, la funzione lavora direttamente sull'originale:

```

void raddoppiaRiferimento(int& x) {
    x = x * 2;    // modifica la variabile originale
}

int main() {
    int numero = 5;
    raddoppiaRiferimento(numero);    // nessun &, si passa direttamente
    cout << numero << endl;        // 10 – è cambiato!
}

```

Il `&` nella **dichiarazione del parametro** dice: "non fare una copia, lavora sull'originale".

Scambiare due variabili

Con i riferimenti, il classico scambio di due variabili diventa molto leggibile:

```

void scambia(int& a, int& b) {
    int temp = a;    // salva a
    a = b;           // a prende il valore di b
    b = temp;        // b prende il vecchio valore di a
}

int main() {
    int x = 10, y = 20;
    cout << x << " " << y << endl;    // 10 20

    scambia(x, y);

    cout << x << " " << y << endl;    // 20 10
    return 0;
}

```

Riferimento costante: leggere senza copiare

Se passi una stringa o un oggetto grande a una funzione, il computer ne fa una copia — spreco di tempo e memoria.

Con `const &` passi il riferimento (nessuna copia) ma impedischi le modifiche:

```

// Senza const ref: copia l'intera stringa (lento per stringhe lunghe)
void stampa1(string s) {
    cout << s << endl;
}

// Con const ref: nessuna copia, non si può modificare (veloce e sicuro)
void stampa2(const string& s) {
    cout << s << endl;
    // s = "altro"; // ERRORE: è const, non si può modificare
}

int main() {
    string testo = "Una stringa molto lunga...";
    stampa1(testo); // fa una copia
    stampa2(testo); // non fa nessuna copia – più efficiente!
}

```

La regola pratica: usa `const string&` (e in generale `const Tipo&`) per passare oggetti grandi in sola lettura.

Riferimenti nel ciclo for

I riferimenti sono utili anche nei cicli `for` quando vuoi modificare gli elementi di una sequenza:

```

int numeri[] = {1, 2, 3, 4, 5};

// Senza riferimento: modifica solo una copia, l'array non cambia
for (int n : numeri) {
    n *= 2; // cambia la copia locale, non l'array
}

// Con riferimento: modifica l'array originale
for (int& n : numeri) {
    n *= 2; // modifica l'elemento vero dell'array
}

for (int n : numeri) {
    cout << n << " "; // 2 4 6 8 10
}

```

Esempio pratico completo

```
#include <iostream>
#include <string>
using namespace std;

// Riceve nome per riferimento costante: nessuna copia, non modificabile
void stampaInfo(const string& nome, int eta) {
    cout << "Nome: " << nome << ", Eta: " << eta << endl;
}

// Modifica il punteggio direttamente
void aggiungiPunti(int& punteggio, int bonus) {
    punteggio += bonus;
}

int main() {
    string giocatore = "Alice";
    int punti = 100;

    stampaInfo(giocatore, 16); // passa il riferimento costante

    aggiungiPunti(punti, 50); // punti viene modificato direttamente
    cout << "Punti aggiornati: " << punti << endl; // 150

    return 0;
}
```

Output:

```
Nome: Alice, Eta: 16
Punti aggiornati: 150
```